

Documento de Requisitos: Sistema de Feedback de Clientes

Versão 2.0

Introdução

O Sistema de Feedback de Clientes é uma solução desenvolvida para coletar, organizar e analisar as percepções dos consumidores sobre os serviços ou produtos de uma organização.

O projeto busca resolver o problema da falta de um canal estruturado para ouvir o cliente, o que muitas vezes resulta em perda de insights valiosos para a melhoria contínua. O desenvolvimento desta solução é importante para centralizar as opiniões, permitindo que a gestão identifique falhas e sucessos de forma rápida e baseada em dados reais.

Público-alvo:

- Clientes: Usuários que desejam expressar sua satisfação ou descontentamento com o atendimento de forma ágil.
- Administradores/Gestores: Responsáveis por monitorar os feedbacks, gerenciar os dados e utilizar as informações para tomadas de decisão estratégicas.

Requisitos Funcionais

ID	Requisito	Descrição	Prioridade
RF001	Coleta de Feedback	O sistema deve permitir que o cliente envie um feedback informando nome, nota (1 a 5) e um comentário.	Essencial
RF002	Validação de Entrada	O sistema deve realizar a validação de campos obrigatórios (nome e nota) antes do envio.	Essencial
RF003	Autenticação Admin	O sistema deve permitir o acesso do administrador a uma página restrita via login simples.	Essencial
RF004	Visualização de Lista	O sistema deve listar todos os feedbacks recebidos em uma tabela na área administrativa.	Essencial
RF005	Exclusão de Dados	O sistema deve permitir que o administrador exclua um feedback da lista.	Importante
RF006	Contador de Registros	O sistema deve exibir um contador total de feedbacks recebidos.	Desejável
RF007	Filtro por Nota	O sistema deve permitir filtrar a lista de feedbacks por nota (ex: exibir apenas notas 5).	Importante
RF008	Notificação de Sucesso	O sistema deve exibir uma mensagem de confirmação de sucesso após o envio do feedback pelo cliente.	Essencial

Requisitos Não Funcionais

ID	Requisito	Descrição	Prioridade
RNF001	Interface Web	O sistema deve possuir uma interface web simples e funcional.	Essencial
RNF002	Persistência	Os dados devem ser armazenados em um banco de dados relacional ou arquivo estruturado (JSON/CSV).	Essencial
RNF003	Compatibilidade	O sistema deve ser executável em navegadores modernos (Chrome, Firefox ou Edge).	Importante
RNF004	Tecnologia	O sistema deve ser desenvolvido utilizando tecnologias web padrão (HTML/JS/CSS e um backend simples).	Essencial

Regras de Negócio

ID	Regra	Descrição	Exceções
RN001	Intervalo de Notas	A nota atribuída pelo cliente deve ser obrigatoriamente um número inteiro entre 1 e 5.	Nenhuma
RN002	Obrigatoriedade de Comentário	O campo de comentário é facultativo, permitindo que o cliente envie apenas a nota.	Nenhuma
RN003	Irreversibilidade	Uma vez que um feedback é excluído pelo administrador, os dados não podem ser recuperados via interface.	Recuperação via backup do banco de dados (se disponível).

Novas Implementações (v2.0)

As funcionalidades abaixo foram implementadas além do escopo original. Todas mantêm o projeto simples de executar localmente.

5.1 Design System — Dark Mode Profissional

O frontend foi construído com um design system completo em CSS puro, baseado em variáveis CSS (tokens) para cores, bordas, sombras e transições. O tema dark mode utiliza uma paleta coesa com destaque em violeta (#6c63ff) e verde-água (#00d4aa).

- Tipografia Google Fonts — família Inter (pesos 300–800).
- Gradientes radiais no hero e nos cards de estatísticas.
- Navbar fixa com backdrop-filter (efeito glassmorphism).
- Responsividade com CSS Grid e a função clamp().

5.2 Animações de Scroll (Intersection Observer)

Todos os elementos de conteúdo recebem a classe .reveal, que inicia com opacity: 0 + translateY(36px). O Intersection Observer adiciona a classe .visible quando o elemento entra no viewport, disparando a animação de fade-in + slide-up. Cards de features têm delay escalonado (nth-child) para efeito cascata.

5.3 Star Rating Interativo — CSS Puro

O seletor de nota usa a técnica CSS de flex-direction: row-reverse combinado com seletores ~ para retroiluminar estrelas anteriores à selecionada. Não utiliza nenhuma linha de JavaScript. Hover adiciona escala (transform: scale(1.2)) à estrela ativa.

5.4 Painel Administrativo com Dashboard

O painel administrativo foi expandido com:

- Sidebar fixa com links de navegação e botão de logout.
- Card — Total de Feedbacks: contador absoluto do banco de dados.
- Card — Média Geral: média aritmética das notas com ícone de estrela.
- Card — Distribuição por Nota: mini gráfico de barras horizontal (CSS puro) mostrando a proporção de feedbacks para cada nota de 1 a 5.
- Badges coloridos na tabela: cada nota recebe uma cor distinta (verde=5, roxo=4, amarelo=3, laranja=2, vermelho=1) com rótulo textual.
- Estrelas visuais na tabela: representação gráfica da nota com estrelas preenchidas e vazias.

5.5 Endpoint de API JSON (Bônus)

Foi adicionado o endpoint GET /api/feedbacks, protegido pela sessão do administrador, que retorna todos os feedbacks em formato JSON. Útil para integração com ferramentas de análise externas ou exportação de dados.

5.6 Preservação de Campos Após Erro de Validação

Ao enviar o formulário com dados inválidos, o backend retorna a página com os valores de nome e comentário preservados, evitando que o usuário precise redigitar tudo. Os erros são listados em um banner detalhado acima do formulário.

Stack Tecnológico

Camada	Tecnologia	Versão mínima
Backend	Python + Flask	Python 3.9 / Flask 3.0
Banco de Dados	SQLite	Biblioteca padrão do Python
Frontend	HTML5 + CSS3 (puro)	—
JavaScript	Intersection Observer API	Navegadores modernos
Tipografia	Google Fonts — Inter	CDN público

Estrutura de Arquivos do Projeto

Arquivo / Pasta	Responsabilidade
app.py	Backend Flask: rotas, lógica, autenticação e queries SQLite
requirements.txt	Dependências do projeto (apenas Flask)
feedback.db	Banco de dados SQLite — criado automaticamente na 1ª execução
static/style.css	Design system completo: tokens CSS, componentes e animações
templates/index.html	Página pública: formulário de envio de feedback com star rating
templates/login.html	Tela de login do administrador
templates/admin.html	Painel admin: dashboard, tabela, filtros e exclusão

Rotas da Aplicação

Método	Rota	Descrição	Acesso
GET	/	Formulário público de feedback	Público
POST	/enviar	Processa e salva o feedback no banco	Público
GET	/admin/login	Tela de login do administrador	Público
POST	/admin/login	Autentica o administrador	Público
GET	/admin/logout	Encerra a sessão admin	Público
GET	/admin	Painel com dashboard, lista e filtros	Admin
POST	/admin/excluir/	Exclui um feedback pelo ID	Admin
GET	/api/feedbacks	Lista todos os feedbacks em JSON	Admin

Modelo de Dados — Tabela feedbacks

Coluna	Tipo	Constraint	Descrição
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador único

Coluna	Tipo	Constraint	Descrição
nome	TEXT	NOT NULL	Nome do cliente
nota	INTEGER	NOT NULL, CHECK (1–5)	Nota de 1 a 5 estrelas
comentario	TEXT	NULL permitido	Comentário opcional
criado_em	TEXT	NOT NULL	Data e hora no formato DD/MM/AAAA HH:MM

Como Executar o Projeto

Obs.: Pré-requisito: Python 3.9 ou superior instalado. Nenhum Docker, banco de dados externo ou arquivo .env é necessário.

Passo	Comando / Ação
1	Abrir o terminal na pasta do projeto: cd "Trabalho QA"
2	Instalar o Flask (apenas na 1ª vez): pip install flask
3	Iniciar o servidor: python app.py
4	Abrir no navegador: http://127.0.0.1:5000

URLs do Sistema

Página	URL	Credenciais
Formulário público	http://127.0.0.1:5000	—
Login administrador	http://127.0.0.1:5000/admin/login	admin / admin123
Painel administrativo	http://127.0.0.1:5000/admin	admin / admin123
API JSON	http://127.0.0.1:5000/api/feedbacks	Sessão admin ativa